

1. Discretización y Estrategia de Implementación Computacional

Para el análisis numérico del sistema dinámico propuesto, se procede a la discretización de las ecuaciones diferenciales ordinarias (EDO) mediante el método de Euler hacia adelante (*Forward Euler*). Este esquema explícito de primer orden aproxima la derivada temporal de una función de estado $f(t)$ a través de un cociente de diferencias finito:

$$\frac{df(t)}{dt} \approx \frac{f(t + \Delta t) - f(t)}{\Delta t} \quad (1)$$

Donde Δt representa el paso de integración. Considerando el dominio temporal discreto $t_n = n \cdot \Delta t$ y denotando el estado de las variables en la iteración n como x_n, y_n, z_n , se establece el siguiente procedimiento matemático y computacional.

1.1. Formulación de las Ecuaciones Recursivas

Partiendo del sistema continuo original (omitiendo los subíndices de neurona individual para generalizar el análisis de una trayectoria), se tiene:

$$\begin{aligned} \frac{dx}{dt} &= y(t) + 3x^2(t) - x^3(t) - z(t) + e \\ \frac{dy}{dt} &= 1 - 5x^2(t) - y(t) \\ \frac{dz}{dt} &= \mu(-z(t) + S[x(t) + 1,6]) \end{aligned} \quad (2)$$

Aplicando la aproximación de Euler y despejando los términos para el instante futuro t_{n+1} , se obtienen las relaciones de recurrencia que gobiernan la evolución del sistema en el algoritmo computacional:

$$x_{n+1} = x_n + \Delta t \cdot (y_n + 3x_n^2 - x_n^3 - z_n + e) \quad (3)$$

$$y_{n+1} = y_n + \Delta t \cdot (1 - 5x_n^2 - y_n) \quad (4)$$

$$z_{n+1} = z_n + \Delta t \cdot \mu \cdot (-z_n + S[x_n + 1,6]) \quad (5)$$

1.2. Parametrización y Condiciones Iniciales

Para la simulación de la dinámica neuronal característica del modelo de Hindmarsh-Rose, se establecen los siguientes parámetros de control reportados en la literatura:

$$e = 3,0, \quad \mu = 0,0021, \quad S = 4,0$$

Se define un paso de tiempo de $\Delta t = 0,001$. Este valor se selecciona como un compromiso óptimo para minimizar el error de truncamiento local inherente al

método de Euler, manteniendo la estabilidad numérica necesaria para capturar tanto las oscilaciones rápidas (*spikes*) como las dinámicas lentas del sistema.

Como condiciones iniciales para el vector de estado $\mathbf{u}_0$ en $t = 0$, se proponen valores que sitúan al sistema en una región próxima al potencial de reposo, garantizando una evolución transitoria observable hacia el atractor:

$$x_0 = -1,6, \quad y_0 = -10,0, \quad z_0 = 0,0$$

1.3. Optimización de Memoria y Configuración del Solver

Dada la necesidad de simular un horizonte temporal extenso con un Δt reducido, el almacenamiento de la totalidad de los puntos calculados ($1/\Delta t$ puntos por unidad de tiempo) resultaría ineficiente, saturando la memoria RAM y el almacenamiento en disco.

Para solucionar esto, se implementa una estrategia de **diezmado** (*downsampling*) en la rutina de guardado. Aunque el sistema se actualiza matemáticamente en cada paso t_n para preservar la precisión numérica, solo se almacenará el estado del sistema para su representación gráfica cada k iteraciones. Se define un factor de diezmado de $k = 10$. De este modo, la escritura en disco se ejecuta únicamente cuando:

$$n \pmod{k} \equiv 0 \tag{6}$$

Para dotar de flexibilidad experimental al código desarrollado, los parámetros críticos de la simulación se organizan en dos niveles:

- **Parámetros físicos del modelo** (constantes globales): Los parámetros e , μ , S y las condiciones iniciales x_0 , y_0 , z_0 se definen como constantes globales, permitiendo modificar el régimen dinámico del sistema sin alterar la lógica del código.
- **Parámetros de discretización** (constantes globales): El paso temporal Δt y el factor de diezmado k se definen también como constantes, garantizando consistencia en la precisión numérica y la estrategia de muestreo.
- **Parámetros de simulación específicos** (argumentos de función): El tiempo de offset t_{offset} y el tiempo total de análisis t_{total} se pasan como argumentos a la función de simulación, permitiendo realizar múltiples experimentos con diferentes ventanas temporales sin recompilar.

Esta arquitectura modular facilita la experimentación sistemática con diferentes configuraciones paramétricas, esencial para el estudio de bifurcaciones y transiciones entre regímenes dinámicos en el modelo de Hindmarsh-Rose.

1.4. Transitorios Iniciales y Ventana Temporal de Análisis

Los sistemas dinámicos no lineales típicamente exhiben un comportamiento transitorio inicial antes de converger a su régimen asintótico característico

(atractor, ciclo límite, comportamiento caótico, etc.). Durante esta fase transitoria, la trayectoria del sistema depende fuertemente de las condiciones iniciales y puede no ser representativa de la dinámica a largo plazo.

Para garantizar que el análisis se centre en el comportamiento genuino del atractor del modelo de Hindmarsh-Rose, se introduce un **tiempo de offset** t_{offset} . Este parámetro define un intervalo temporal inicial durante el cual se ejecuta la integración numérica, pero *no* se registran los datos para posterior análisis. Matemáticamente:

$$\text{Datos guardados} = \{(t_n, x_n) : t_n \geq t_{\text{offset}} \text{ y } n \equiv 0 \pmod{k}\} \quad (7)$$

En la implementación computacional se utiliza $t_{\text{offset}} = 1000$ unidades de tiempo, lo que permite que el sistema alcance su estado estacionario antes de iniciar la recolección de datos. Posteriormente, se captura una ventana temporal de $t_{\text{total}} = 4000$ unidades de tiempo, suficiente para caracterizar la dinámica periódica o cuasi-periódica de la actividad neuronal.

1.5. Persistencia de Datos y Visualización

La arquitectura computacional implementada separa claramente dos fases del análisis numérico:

1. **Fase de simulación:** La función de simulación ejecuta el bucle de integración temporal completo ($t_{\text{offset}} + t_{\text{total}}$ unidades de tiempo) y almacena los datos diezmados directamente en un archivo de texto estructurado en formato CSV (*Comma-Separated Values*). Este enfoque de escritura incremental evita la acumulación de vectores extensos en memoria RAM.
2. **Fase de visualización:** Una función independiente lee el archivo generado y construye la representación gráfica de la serie temporal $x(t)$. Esta modularización permite reutilizar los datos sin necesidad de repetir simulaciones computacionalmente costosas, facilitando además el análisis posterior mediante herramientas externas.

La salida gráfica representa la evolución temporal de la variable $x(t)$, que en el contexto del modelo de Hindmarsh-Rose corresponde al **potencial de membrana** de la neurona. Esta señal captura tanto los eventos rápidos de disparo (*spikes*) como las modulaciones lentas asociadas a la variable de adaptación $z(t)$, características distintivas del *bursting* neuronal.